DeepPerturb Reflection

Zefeng Xu, David Naranjo, Moses Yang, Minh Le Tran
GitHub: https://github.com/DavidNaranja/final_proj_CPA/tree/main

Title:
Interpretable Prediction of Single Cell Gene Expression After Perturbation

Introduction:
With current high-throughput technology, researchers are able to record the exact effects of drugs and other conditions on cells through changes in their gene expression. Such data has potential in accelerating drug development. However, the immense cost and scale of creating a complete database of cell response through physical experimentation makes computational methods essential. The subsequent challenge is to predict cellular response to novel conditions using knowledge of previous experiments.

The paper we chose to reimplement introduces a novel deep learning framework called the Compositional Perturbation Autoencoder (CPA). Its core objective is to enable *in silico* simulations of cellular responses in conditions that are difficult or infeasible to measure experimentally, such as unseen drug combinations or dosages.

We chose this paper because it addresses a fundamental limitation in single-cell perturbation studies: the inability to experimentally test every combination of perturbations, cell types, and time points. CPA's compositional and modular architecture makes it both powerful and generalizable, and we were particularly interested in reimplementing it to better understand how machine learning models can support counterfactual reasoning in biology and accelerate hypothesis generation for drug discovery and gene function analysis.

This CPA model is, in summary, a supervised, structured regression with conditional generation. CPA is a supervised conditional generative model, trained to reconstruct single-cell gene expression given known perturbations and covariates.

Methodology:
Dataset:
We used a publicly available dataset from the paper containing the scRNA data of PBMC cells under the effects of IFN-β. scRNA data describes cell state as an observation of how many times each gene has been read, specifically the number of mRNA transcripts corresponding to genes at the time of measurement.

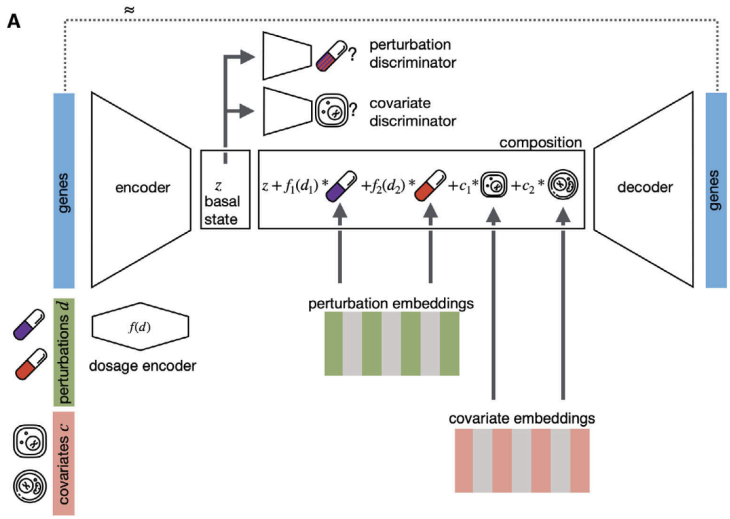
Across many cells, this following forms the counts matrix. Our dataset contains 13576 single cells and after extraction of highly variable genes, there remains 5000 genes in the count matrix.

	Gene 1	Gene 2
Cell 1	5	2
Cell 2	1	3

The dataset additionally contains a quantification of the exact conditions the cells were under. The conditions are split into covariates, describing cell-specific information like cell type and experimental batch, and perturbations, indicating direct influences on the cell, which is simply whether IFN-β was applied to the cell or the cell was a control in this dataset. We used the scRNA, covariate, and perturbation data of all cells in the dataset, ignoring additional metadata contained within.

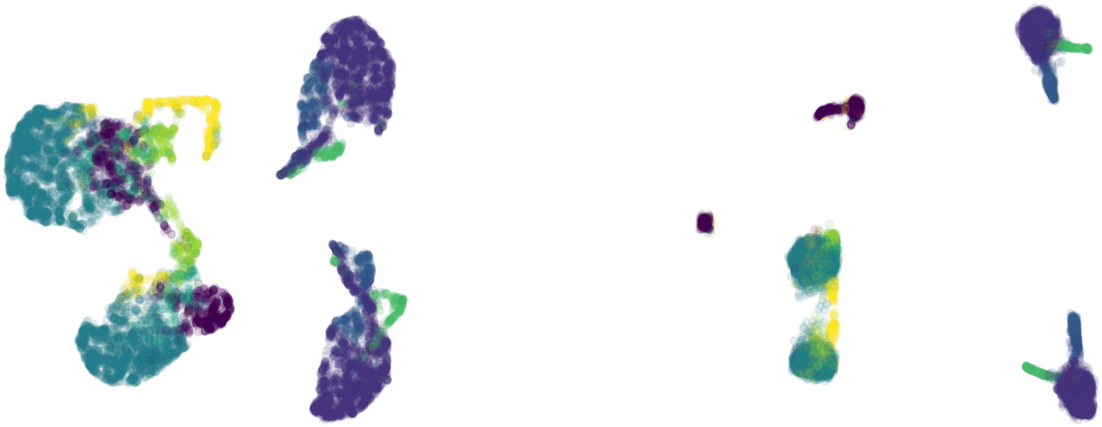
Model:

The CPA model is a conditional autoencoder that takes in the perturbed cell state (observed gene expression), covariates, and perturbations while using adversarial learning to create embeddings representing a cell's theoretical basal state. Through the encoding process, the cell state, covariates, and perturbations are each embedded separately. Through the use of the two discriminators, the effects of the covariates and perturbations are separated from the cell state, leading to what should be an embedding representing the basal cell state. Consequently, two identical cells under different perturbations should theoretically be embedded to the same point in latent space. The three embeddings of basal state, covariates, and perturbations are then combined through simple addition, with the perturbations scaled by doses (another nonlinear function to learn through training) and decoded into a reconstruction of the initial cell state. Optimization of the network comes from minimizing the reconstruction loss and the adversarial loss. Below is a roadmap of the model architecture.



During training, the dataset was split into a training set, a validation set, and a testing set, where the testing set contains solely all the samples that belong to a specific cell type (i.e., OOD testing set). To generate predictions of the testing set, the basal state was combined with new perturbation embeddings, yielding predictions of the cell state under new perturbation conditions. In training, the loss functions used were Mean Squared Error for reconstruction and Sparse Categorical Cross Entropy for the discriminators; the optimizers used were both Adam but with different learning rates for reconstruction and discriminators.

Results:
We measured the performance of our model primarily on its ability to remove the covariate and perturbation effects, embedding the input cell state as the basal cell state, because this is the very first thing the CPA model should theoretically be able to achieve. To measure this, we retrieved the embedded cell state (basal state) of each cell and plotted the first two UMAP dimensions against the first two UMAP dimensions of the original cell state (input to the encoder). The ideal results would be complete dispersion of each point (no clustering) as removal of covariate and perturbation effects would leave no differences against cells of the same type. Our results (left) only show moderate dispersion compared to the original (right). However, this already shows that the model was taking effect.



We additionally measured the performance of our model using the R-square score, comparing the model's predicted gene expression of the testing set with ground truth values. However, we have not been able to achieve good results as of today and this has been one of the primary challenges we were facing.

Challenges:
The initial creation of the model — obtaining data, preprocessing data, and building the general model architecture — based on the implementation described in the paper went smoothly enough. There was an issue implementing the negative gaussian likelihood loss function featured in the paper given the differences between pytorch and tensorflow, but we believe it's unlikely to have caused the performance issues we later faced. We struggled with the model underperforming compared to the authors' implementation. Another challenge was the time it took to run the

model on large datasets, preventing us from easily tuning hyperparameters and trying different architectures. We suspect that these challenges are relatively connected; given more time and quicker runtime, we would have been able to run hyperparameter optimization techniques like grid search. Besides hyperparameter tuning, we would have also liked to try different activation functions, varying amounts of layers, and other architectural designs, especially the balancing between the two loss functions and optimizers.

Reflection:
We feel that the project ultimately turned out to be successful, given our goals. We accomplished our base goal of converting from PyTorch to TensorFlow to implement the model and using our preprocessed data to train the model. Given the time constraints, we continued to work on our target goal up until final submission, which was to get predictions and tune the hyperparameters to improve the model's predictions, ideally to the extent of the paper's implementation. While we were not able to achieve the performance shown in the original paper, we were able to obtain significant results with our hyperparameters. We were not able to accomplish our stretch goal, which was to build additional state-of-the-art models to highlight the efficacy of the CPA architecture.

We went back and forth between a TensorFlow implementation and a PyTorch implementation. We expected Brown's supercomputer, Oscar, to perform better with PyTorch, given its flexible, dynamic and low-level nature, so we initially planned to implement the model in PyTorch. However, we ultimately decided to work with TensorFlow because it was the library that was used in class and the library that our group was most familiar with. It was important for us to have conversations early on in the timeline of the project to ensure that we were all on the same page and agreed on our implementation. Given the scope and time constraints of the project, we do not believe that we would have made any changes, were we to do our project over again.

Given more time, we would want to experiment with various hyperparameters, activation functions, loss functions, and evaluation metrics to optimize the performance of our model. As a specific example, the paper used Gaussian negative log-likelihood (in combination with an adversarial approach to loss) as the reconstruction loss function. In contrast, we used a simple Mean Squared Error as the reconstruction loss function. The paper's implementation of the Gaussian negative log-likelihood was more concise with PyTorch, so we would like to transfer this implementation to TensorFlow, if given the time. Furthermore, we would want to spend more time balancing the learning rates of the encoder and the discriminators. If we were able to find an ideal balance, the model would be able to learn as pure a basal state as possible, allowing for a better practical application of the model.

One of the biggest takeaways from this project was the realization that oftentimes, a hybrid approach to certain problems are the most successful. In the case of this project, we saw that traditional Deep Learning methods were often difficult to scale and had trouble with combinations of perturbations. It was interesting to learn about the success of the paper's implementation of

CPA, which involved an architecture containing both a Variational Autoencoder and Adversarial Learning. Another takeaway from this project was the importance of hyperparameter tuning. In the assignments that we did in class, the majority of the work was focused on understanding and building the architecture of different models. In general, we only needed to tune hyperparameters to meet the specific accuracy requirements determined by the assignment itself. However, it is difficult to determine where “good enough” stands, so it is also difficult to know how to tune hyperparameters, as well as know when to stop.